Widget Controls

WIDGET API VERSION 1.4.0

Widget controls are meta parameters that enable users to customize widget behavior and appearance through the iCUE settings panel. Each control is defined as a `<meta>` tag in the widget's HTML and becomes a global JavaScript variable accessible in your widget code.

Quick Start

Define a control using a meta tag with the `x-icue-property` name:

```
<meta name="x-icue-property" content="textColor" data-label="Text Color" data-type="color" data-default="'#FFFFFF'" />
```

Access the value in JavaScript:

```
console.log(textColor); // "#FFFFFF"
document.body.style.color = textColor;
```

Required Attributes

Every control must include these attributes:

Attribute	Description	Example	Support JS expressions
`name`	Must always be `"x-icue-property"` (constant value)	`"x-icue-property"`	✗

Attribute	Description	Example	Support JS expressions
`content`	JavaScript variable name. Must be unique and use only alphanumeric characters and underscores	`"fontSize"`	❌
`data-label`	User-facing label shown in iCUE settings panel	`"Font Size"`	✅
`data-type`	Control type (see Available Control Types)	`"slider"`	❌

> **VARIABLE NAMING**
>
> The `content` attribute defines the JavaScript variable name. Use descriptive camelCase names like `textColor`, `fontSize`, or `showIcon`.

Dynamic Values with JavaScript Expressions

All `data-*` attributes (except `data-type`) support JavaScript expressions for dynamic behavior. You can use:

- **Standard JavaScript** – `Math`, `String`, `Array`, `Date`, etc.
- **iCUE Global Object** – System information via `iCUE` object
- **Plugin Data** – Sensor values, media info, etc. via plugins

Example: Dynamic default based on system locale

```
<meta
    name="x-icue-property"
    content="temperatureUnit"
    data-label="Temperature Unit"
    data-type="combobox"
    data-default="iCUE.defaultTemperatureUnit()"
    data-values="['°C', '°F']"
/>
```

Example: Localized labels

```
<meta
    name="x-icue-property"
    content="theme"
    data-label="tr('Theme')"
    data-type="combobox"
    data-values="[
        {'key': 'light', 'value': tr('Light')},
        {'key': 'dark', 'value': tr('Dark')}
    ]"
/>
```

Available Control Types

Basic Input Controls

Control	Description	Use Case
slider	Numeric slider with min/max range	Opacity, font size, refresh rate
switch	Boolean toggle (on/off)	Enable/disable features
textfield	Single-line text input	Custom text, API keys, URLs
color	Color picker with hex values	Text color, background color, accent color

Selection Controls

Control	Description	Use Case
combobox	Dropdown selector	Predefined options, themes, layouts
search-combobox	Searchable dropdown with filtering	Long lists, timezone selection
tab-buttons	Tab-style button group	Visual mode selection (2-4 options)

Media & Sensors

Control	Description	Use Case
media-selector	File picker with image/video support and transformations	Background images, logos, animations
sensors-combobox	Hardware sensor selector	CPU temp, GPU usage, fan speed
sensors-factory	Multiple sensor configuration with custom colors	Multi-sensor displays

How Controls Work

1. **Definition** – Controls are defined as `<meta>` tags in your widget's `<head>` section
2. **Organization** – Group related controls using property groups
3. **Injection** – iCUE automatically creates global JavaScript variables for each control
4. **Updates** – When users change values, iCUE triggers the `onDataUpdated` event
5. **Access** – Read current values directly from the global variables in your widget code

Example lifecycle:

```
<head>
    <meta
        name="x-icue-property"
        content="refreshRate"
        data-label="Refresh Rate"
        data-type="slider"
        data-default="30"
        data-min="1"
        data-max="60"
        data-step="1"
        data-unit-label="'Hz'"
    />

    <script type="application/json" id="x-icue-groups">
        [{ "title": "Settings", "properties": ["refreshRate"] }]
    </script>
</head>
<body>
    <div id="display"></div>
    <script>
        icueEvents = {
            onICUEInitialized: init,
            onDataUpdated: update,
        };

        function init() {
            update();
            startRefreshLoop();
        }

        function update() {
            // Access the current value
            document.getElementById("display").textContent = `Refresh:
${refreshRate}Hz`;
        }

        function startRefreshLoop() {
            setInterval(() => {
                // Use refreshRate value to control behavior
                fetchData();
            }, 1000 / refreshRate);
        }
    </script>
</body>
```

Best Practices

Naming Conventions

✅ Good variable names:

- `textColor`, `backgroundColor` – Clear and descriptive
- `showWeatherIcon`, `enableAnimations` – Boolean intent is clear
- `refreshInterval`, `maxItems` – Purpose is obvious

❌ Avoid:

- `color1`, `color2` – Ambiguous purpose
- `setting` – Too generic
- `data` – Unclear meaning

Label Guidelines

✅ Good labels:

- "Text Color" – Clear and concise
- "Refresh Interval" – Describes what it controls
- "Show Weather Icon" – Action is clear

❌ Avoid:

- "Color" – Too vague (which element?)
- "Setting 1" – Meaningless to users
- "The color of the text displayed on screen" – Too verbose

Default Values

- Provide sensible defaults for all controls
- Use string literals for text values: `data-default="'#FFFFFF'"`
- Use numbers directly: `data-default="100"`
- Consider system preferences with `iCUE` functions

Next Steps

- Learn about Property Groups to organize controls in the settings panel
- Explore JavaScript Expressions for dynamic control behavior
- Understand the Event Lifecycle to react to control changes
- Review individual control documentation for specific attributes and examples